2023 10th International Conference on Advanced Informatics: Concept, Theory and Application (ICAICTA) | 979-8-3503-2991-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICAICTA59291.2023.10389879

Analyzing Fair Share Fairness of Tasks in the Linux Completely Fair Scheduler using eBPF

[1]Jesson Yo, [2]Achmad Imam Kistijantoro
School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Bandung, Indonesia
[1]jessonyo@students.itb.ac.id, [2]imam@itb.ac.id

***Abstract*—The Linux Completely Fair Scheduler (CFS) was designed by the means of allocating CPU resources appropriately based on a certain notion of fairness. Fairness in the context of Completely Fair Scheduler refers to the proportion of CPU time (fair share) each thread/process (task) can use based on its priority level. However, most of the time the actual fair shares that each task receives are different compared to the fair shares dictated by the priority levels, which might lead to unwanted system behaviors. We present a system to monitor the Completely Fair Scheduler – Durian – that utilizes eBPF (extended Berkeley Packet Filter) to emit certain scheduling events and data toward user space which among many things will analyze the actual fairness of fair shares each task gets based on scheduling data and statistics. The system provided, although uses a certain amount of time and resources, will have little to no hindrance to the Linux scheduling algorithm in terms of time and safety.**

Operating Systems, Process Scheduling, Task Scheduling, Completely Fair Scheduler, Fairness, Linux, eBPF

I. Introduction

Completely Fair Scheduler (CFS) is one of the Linux scheduling policies that was designed to divide CPU time among a group of runnable tasks from the fair class based on a certain notion of fairness. This fairness ensures a level of balance between throughput and interactiveness. One such example is allowing CPU-bounded and I/O-bounded processes to share the same set of resources in an ideal manner through scheduling. Processes that are CPU-bound will receive larger chunks of CPU time at once and have a minimal amount of context switching. On the other hand, I/O bounded processes will receive smaller chunks of CPU time but it will get scheduled often hence a big amount of context switching. This scheduling policy is realized by assigning priority values for each runnable task.

Priority dictates the proportion of CPU time (fair share) that each task receives. High-priority tasks are tasks with small numerical priority values and are usually reserved for CPU-bounded processes as a way to give a bigger proportion of the CPU and vice versa for I/O-bounded processes. However, each task's actual fair share is most likely to be different compared to the theoretical fair share that is calculated purely using priority. The reason for this is because CPU-bounded and I/O-bounded processes can't be defined using priority values alone. Priority only dictates the amount of time each task can use to execute in the CPU but not how often it gets scheduled where the latter is important for interactiveness. CFS has certain mechanisms and heuristics outside of just task priority to achieve this.

While these heuristics are often beneficial for directly manipulating which tasks get executed and for how long, they also affect the fair share that is given to each task. These heuristics and mechanisms are not the only sources of unfairness in CFS. Even if those heuristics are removed, CFS can also cause fairness issues depending on how fairness is defined. Fairness issues can happen in a multi-threaded environment as the Linux scheduler tends to favor programs with a higher number of threads [1] if fairness is viewed at the process level instead of at a thread level.

An example of an unfair fair share is when a task wants to do a blocking I/O operation. Normally the task will ask the kernel to change its state to either TASK_INTERRUPTIBLE or TASK_UNINTERRUPTIBLE and do a *sched_yield*, relinquishing its right of the CPU proportion prematurely and finally getting dequeued from the run queue. This on its own is fair because the virtual runtime of the task grows as much as the actual CPU time that is used. When a task stops being executed in the CPU in a non-preemptive manner, the virtual runtime only grows as much as it needs. The unfairness comes from the enqueuing algorithm which happens after the task finishes its I/O operation. The virtual runtime of the task is set to the minimum virtual runtime from the run queue. This is a heuristic used by CFS to circumvent starvation by avoiding setting a newly woken-up task with a low virtual runtime value. Without this heuristic, if the newly woken-up task has a very low virtual runtime far below the minimum virtual runtime, it will dominate the CPU until its virtual runtime matches the minimum virtual runtime. It is quite obvious that this heuristic can reduce the fair share that a task should get because it can increase the virtual runtime of tasks that do I/O often above the actual virtual runtime it should get hence reducing its CPU time.

In this paper, we present the design of Durian and the evaluation of its current implementation especially on its impact on the performance of the system. Durian is an eBPF-based scheduler monitoring system that keeps track of task states that are scheduled using CFS. The key challenge of building a system based on eBPF is due to the constrained

programming model of eBPF and that is to find a balance between which parts of the program should be offloaded to user space and which part should be kept in the kernel space [2].

Durian does this by exporting certain scheduling events from kernel space to user space decorated with context-specific information such as event timestamp. This information is then used to derive task states and statistics in user space. Scheduling statistics for each task are used to determine how fair the share that each task receives. It is done by comparing them to the theoretical fair shares that are dictated by the task weight and the run queue weight load.

The design goal of Durian is to decouple the data collection process (the process of exporting events from kernel space to user space and deriving task statistics) and the analysis process. The reason for this design is to allow different methods of analysis from the statistics that are collected by keeping it generic. Statistics are stored in an external store so that other processes, possibly more than one, can do their independent analysis of the data. We also want Durian to have little to no impact on the performance of the scheduling algorithm and its safety.

Our insight is that the delta between the theoretical and actual fair share can be utilized to know how much time quantum a task relinquished/received throughout its lifecycle. Since we know the actual fair share that is lost or gained, we can tune parts of our system to determine whether we want to lose or gain even more time quantum to achieve our goals. Using Durian, it is easy to extract this information. On top of that Durian is customizable, giving engineers freedom for which aspect they want to be analyzed instead of relying on specific constrained data that is exposed automatically by the kernel.

II. Background

A. Linux Task Scheduling

The CPU scheduling algorithm in the Linux kernel has been researched many times in multiple aspects and has been reworked over the years. Two of the most recent iterations are called the O(1) scheduler [3] and the Completely Fair Scheduler (CFS) [4]. Schedulers are normally designed to achieve a certain level of interactivity and throughput. CFS wants to model a perfectly multitasking system, maximizing throughput through CPU utilization while maximizing interactiveness where both of these are considered to be contradictory to each other. It is not just CFS, most of the attention in recent years has been focused on designing a scheduling algorithm that balances interactivity and scalability with a Simultaneous Multi-Processing (SMP) system [5].

Although the official documentation of the Linux scheduler has mentioned the fact that CFS doesn't have any heuristics, at least based on one of the long-term versions that is used at the time of writing which is version 5.10.xxx, this is simply not the case. Internally CFS uses a red-black tree to manage the list of runnable tasks and retrieve the task with the lowest vruntime. To choose which task deserves to be executed in the CPU, it simply chooses the task with the lowest vruntime from the tree, executing and incrementing the vruntime value until it is no longer the task with the lowest vruntime and reselecting another task with a smaller vruntime.

The growth rate of the vruntime is directly proportional to the length of the latest CPU burst and the task weight which is derived from task priority. A higher weight means the vruntime grows slower compared to another task with a smaller weight assuming both of them have the same runtime. This allows tasks with higher weight to use a bigger proportion, also called fair share, of the CPU time. Tasks that have higher priority will have a higher weight to allow them to get more shares of the CPU and vice-versa. To avoid confusion, the definition of a high-priority task is a task with a small numerical value of task priority. In the context of tasks that are scheduled using CFS, that refers to tasks with priority values that are approaching 100 and tasks with low priority refers to tasks with priority values that are approaching 139 as CFS (SCHED_NORMAL) only deals with processes with the priority range of 100-139.

The core CFS algorithm we just described doesn't use any heuristics if it works as described. It is things like vruntime adjustment when waking up a task, sched_yield, and other capabilities that are there for good reason that should be considered as heuristics. Not to mention features have been added continuously to improve the CFS algorithm and although it is impractical to review each of them, it is worth noting some of these are heuristics. The reason we care about these heuristics is that they are impacting the notion of fairness that was first proposed and modelled after. Giving a fair share penalty for a task that only does I/O-related workload is desirable as the computation needed is almost non-existent and we want the task to be responsive and not hinder other tasks that need higher computational resources. On the other hand, giving penalties to a certain type of task, like a database where it might do a high amount of I/O, is undesirable, as we know a lot of the time it has to use a significant amount of CPU for heavy transactions processing among many other things.

Whether their fairness is compromised for better or for worse is highly dependent on the workload but still, it is advantageous to be able to know the fair share a task receives in reality and the fair share that it should receive theoretically without any heuristics. Durian will keep track of the task states and derive necessary statistics in an external repository to be analyzed. Here we are interested in the fairness of the share that each task receives.

B. eBPF

eBPF is an extension of the initial new kernel architecture for packet capture, BPF [6]. BPF allows customizable packet filtering applications to be injected into the kernel space from

user-space. In this day and age, BPF has grown more mature and now is called eBPF. It has been used in more than just packet filtering and has generic functionalities. The definitions of eBPF vary, but the most representative of all is a *mechanism to execute user space code in kernel space in an event-driven manner.*

This event-drivenness provides the ability to create an on-demand monitoring tool that can be executed in kernel space. By being event-driven, this means the eBPF program will only get executed when needed hence its consumption of resources is efficient [7]. Among many other things, for performance monitoring tools, this allows eBPF to have a completely decoupled functionality from whatever it is trying to probe.

eBPF has a strict verification process that enumerates all branches of conditional and loop statements to make sure every execution meets the safety and liveness requirements [8]. Because of this, there can only be up to 1 million instructions in an eBPF program, which can be bypassed using tail calls. It also prohibits dynamically allocated memory, instead, it uses eBPF maps with statically specified capacity that lives in the locked memory region to avoid page faults. eBPF can only borrow certain functionalities from the kernel by the use of BPF helper calls [9], so it doesn't have full access to the kernel source.

CPU tracing tools can be built on top of eBPF. Statistics can be derived using eBPF with the help of tracepoints and kprobe instrumentations [10]. Tracepoints for scheduler and syscall events while kprobe for scheduler internal functions that are not exposed using tracepoints. Although BPF programs are small and highly efficient, executing them during each context switch could lead to a cumulative overhead that becomes noticeable or even substantial. In the most extreme scenario, scheduler tracing might impose a burden of more than 10% on the system's performance. Without proper optimization, this overhead would be unacceptably high [10].

III. System Design and Architecture

Durian consists of three parts which are the probes, the daemon process to manage the probes, and finally the analyzer. Durian probes are simply eBPF programs that are attached to scheduler tracepoints to detect certain events and capture the context data to be exported to user space through a ring buffer type eBPF map [11]. The ring buffer will behave as a multi-producer single-consumer message queue and preserve the events ordering. This data will be consumed by the daemon process and it will also do the necessary computation for the sake of deriving tasks statistics. Redis is used as the external store for ease of data access by the data analyzer(s). An illustration of the entire system architecture is shown in Figure 1.

The eBPF programs are referred to as Durian Probes in the following sections. Their function is to listen to certain scheduling events. Durian Daemon is used to make reference to the daemon process shown in Figure 1 while Durian Analyzer is used to speak about the data analyzer.

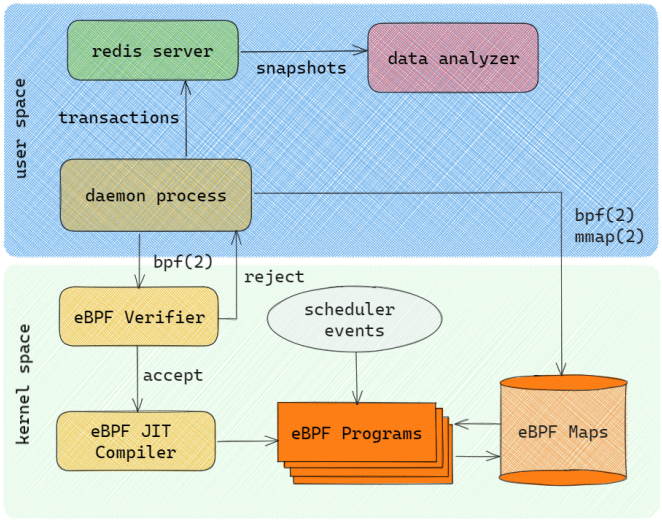


Figure 1. Durian system architecture

A. Durian Probes

This component is composed of four eBPF programs that are attached to exactly one different scheduler tracepoint each. Every program is responsible for giving task state change information from kernel space to user space. The information is wrapped and sent in a custom C struct to pass context data along with the task state information. The four tracepoints that we are interested in are sched_wakeup_new, sched_wakeup, sched_switch, and sched_process_exit. Each tracepoint tells Durian when a certain task is first initialized, woken up from a wait queue, did a context switch, and did a process exit. Each probe will also pass the kernel timestamp to indicate when a certain event happens among other context-specific data that is also passed to user space.

An eBPF map with the type of ring buffer is used to pass the data to user space to be consumed by the Durian client. The ring buffer is convenient because every CPU will have access to the same singular ring buffer, hence preserving the global ordering for the events becomes trivial [12]. Nowadays it is a common pattern to use an eBPF ring buffer to synchronize consumer/producer access [13]. It is possible for different eBPF and user space programs to access a shared eBPF map [14], in this case, a ring buffer consequently giving global ordering of all event types.

This is different from an older approach using perf events array map as every CPU core will have access to a different array map [14] resulting in a significant effort and complexity to know the ordering of events across CPUs. Not even the kernel timestamp is reliable as the source of truth to sort the

events into a specific ordering as the CPU clock is not guaranteed to be synced across different cores [15].

B. Durian Daemon

The daemon process is used to load the eBPF programs and the eBPF map to kernel space. Almost every eBPF program will have a corresponding user space program, Durian daemon is responsible for being the user-space side of Durian probes. It is also responsible for checking all necessary preparation has been finished successfully including checking whether a Redis instance is ready for data collection and whether all necessary Redis lua transaction scripts have been loaded and cached by Redis.

Durian daemon will do polling to the ring buffer to consume the data entries. Because the data that is stored in the ring buffer contains the kernel timestamp, it is possible to derive time statistics in conjunction with the task's last seen state which is stored in Redis. Using both of those information we can figure out the time delta between two events, for example, the time delta between when a task is context switched into the CPU and when a task is context switched out of the CPU can be used to determine the duration of the last CPU burst time.

C. Durian Analyzer

The Durian analyzer refers to any external process that makes use of the data that is stored in the Redis server and does certain derivations to analyze a certain aspect of the scheduling process. It works by taking a snapshot of the data from Redis and storing it in a bin code format for further analysis. Since we care about the fair share fairness of the tasks, the data analyzer will print out the necessary information that can be used by the user to determine the fairness and view the statistics of the task. It will give information about the statistics, analysis, and configuration used.

The fair share fairness of the tasks can be seen in Figure 3, specifically using the columns fair_ns and ideal_fair_ns. These values are derived from the raw statistics displayed in Figure 2. The configuration values used during the derivation are also shown in Figure 4 for completeness as different values can affect the values in Figure 3. Fair ns is the actual measured fair share that each task receives throughout its lifetime, normalized to sched_latency_ns. While ideal fair ns is defined as the theoretical fair share the task should receive for a certain time period (epoch or sched_latency_ns) if the scheduling policy only considers the task priority and ignores any heuristics. It follows Equation 1. The definition of fairness here is defined as how small the difference between ideal fair ns and fair ns. Here $weight_i$ and $timeslice_i$ refer to a certain task's weight and timeslice result while the sum of all weight in the denominator is the sum of all running tasks' weight.

$$timeslice_i = \frac{weight_i}{\sum_{j=0}^{n-1} weight_j} \cdot sched\_latency\_ns$$

Equation 1. ideal fair ns formula

The difference can be further exacted by calculating the actual priority level that each task exhibits. This actual priority level, we call fair_share_prio. It is the priority level that will correspond to the fair_ns that each task receives if we assume that all tasks are CPU-bound and use their fair share completely. It is hard to measure the exact priority value for each task as it is a function of n variables where n is the number of tasks, hence we introduced a heuristic function to calculate the delta priority/delta nice which makes use of a decay factor k which is the average time decay factor that is derived from the kernel source code.

$$\Delta priority = round(log_k(fair\_ns) - log_k(ideal\_fair\_ns))$$

Equation 2. delta priority formula

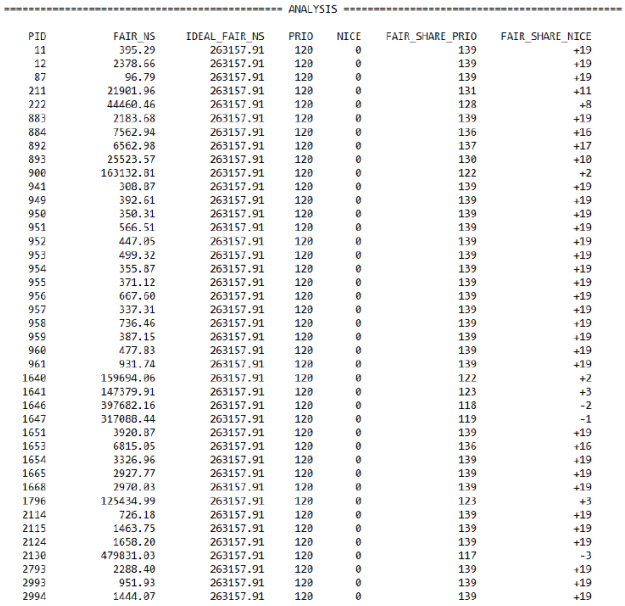
The value is then clamped to be between -20 and +19 inclusive as that is the priority range of SCHED_NORMAL tasks in the Linux kernel. A higher (more positive) delta priority value indicates that the task has lost more time quantum, while conversely, a lower (more negative) delta priority value suggests the task has gained time quantum.

```
Tasks:     76 total,     1 on cpu,     11 on run queue,     61 waiting,     3 stopped
Average I/O: 8519102500 ns,      average CPU: 184058830 ns

========================================= STATISTICS =========================================
```

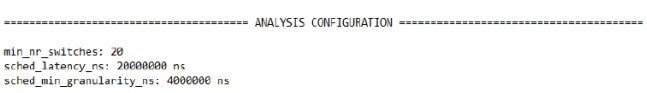
PID	PRIO	NICE	TIME+	COMMAND	TOTAL_CPU(NS)	TOTAL_WAIT(NS)	NR_SWITCH
11	120	0	00-00:00:06	ksoftirqd/0	188024	6399479928	22
12	120	0	00-00:00:10	rcu_sched	1780332	10065766530	434
87	120	0	00-00:00:09	kcompactd0	69850	9709782079	20
211	120	0	00-00:00:09	node	16238865	9958450015	93
222	120	0	00-00:00:09	node	32232372	9720624066	208
883	120	0	00-00:00:09	init	1618862	9971722666	52
884	120	0	00-00:00:09	node	5606793	9968339512	52
892	120	0	00-00:00:09	init	4789358	9812399137	145
893	120	0	00-00:00:09	node	18625801	9798318239	145
900	120	0	00-00:00:09	node	113559036	9250864125	223
941	120	0	00-00:00:09	node	218285	9508242779	20
949	120	0	00-00:00:10	cpptools	291868	10001480763	31
950	120	0	00-00:00:08	cpptools	219892	8444909492	24
951	120	0	00-00:00:10	cpptools	421142	10001361248	25
952	120	0	00-00:00:10	cpptools	332334	10001417687	28
953	120	0	00-00:00:08	cpptools	313429	8444924992	24
954	120	0	00-00:00:08	cpptools	223385	8444960080	26
955	120	0	00-00:00:10	cpptools	275890	10001396893	28
956	120	0	00-00:00:08	cpptools	419053	8444798217	28
957	120	0	00-00:00:10	cpptools	250760	10001514564	28
958	120	0	00-00:00:10	cpptools	547487	10001257107	28
959	120	0	00-00:00:10	cpptools	287810	10001473841	30
960	120	0	00-00:00:10	cpptools	355217	10001466686	31
961	120	0	00-00:00:10	cpptools	692646	10000914398	50
1640	120	0	00-00:00:09	wsl-bootstrap	107219303	8898399663	4593
1641	120	0	00-00:00:09	wsl-bootstrap	108710924	9665232819	30227
1646	120	0	00-00:00:04	wsl-bootstrap	137319146	4470600767	6132
1647	120	0	00-00:00:09	wsl-bootstrap	231262548	9520799182	9635
1651	120	0	00-00:00:09	init	2911428	9986671544	127
1653	120	0	00-00:00:10	vpnkit-bridge	5068051	9994625701	1210
1654	120	0	00-00:00:09	vpnkit-bridge	2395899	9686274697	57
1665	120	0	00-00:00:09	vpnkit-bridge	2108406	9686483530	54
1668	120	0	00-00:00:09	vpnkit-bridge	2027155	9180650116	53
1796	120	0	00-00:00:09	wsl-bootstrap	85019019	9010637020	3590
2114	120	0	00-00:00:09	containerd	531849	9853018425	66

Figure 2. Durian analyzer tasks statistics output



```
============================================ ANALYSIS ============================================
  PID          FAIR_NS     IDEAL_FAIR_NS    PRIO    NICE    FAIR_SHARE_PRIO    FAIR_SHARE_NICE
   11            395.29          263157.91     120       0                139                +19
   12           2378.66          263157.91     120       0                139                +19
   87             96.79          263157.91     120       0                139                +19
  211          21901.96          263157.91     120       0                131                +11
  222          44460.46          263157.91     120       0                128                 +8
  883           2183.68          263157.91     120       0                139                +19
  884           7562.94          263157.91     120       0                136                +16
  892           6562.98          263157.91     120       0                137                +17
  893          25523.57          263157.91     120       0                130                +10
  900         163132.81          263157.91     120       0                122                 +2
  941            308.87          263157.91     120       0                139                +19
  949            392.61          263157.91     120       0                139                +19
  950            350.31          263157.91     120       0                139                +19
  951            566.51          263157.91     120       0                139                +19
  952            447.05          263157.91     120       0                139                +19
  953            499.32          263157.91     120       0                139                +19
  954            355.87          263157.91     120       0                139                +19
  955            371.12          263157.91     120       0                139                +19
  956            667.60          263157.91     120       0                139                +19
  957            337.31          263157.91     120       0                139                +19
  958            736.46          263157.91     120       0                139                +19
  959            387.15          263157.91     120       0                139                +19
  960            477.83          263157.91     120       0                139                +19
  961            931.74          263157.91     120       0                139                +19
 1640         159694.06          263157.91     120       0                122                 +2
 1641         147379.91          263157.91     120       0                123                 +3
 1646         397682.16          263157.91     120       0                118                 -2
 1647         317088.44          263157.91     120       0                119                 -1
 1651           3920.87          263157.91     120       0                139                +19
 1653           6815.05          263157.91     120       0                136                +16
 1654           3326.96          263157.91     120       0                139                +19
 1665           2927.77          263157.91     120       0                139                +19
 1668           2970.03          263157.91     120       0                139                +19
 1796         125434.99          263157.91     120       0                123                 +3
 2114            726.18          263157.91     120       0                139                +19
 2115           1463.75          263157.91     120       0                139                +19
 2124           1658.20          263157.91     120       0                139                +19
 2130         479831.03          263157.91     120       0                117                 -3
 2793           2288.40          263157.91     120       0                139                +19
 2993            951.93          263157.91     120       0                139                +19
 2994           1444.07          263157.91     120       0                139                +19
```

Figure 3. Durian analyzer analysis output



```
====================================== ANALYSIS CONFIGURATION ======================================
min_nr_switches: 20
sched_latency_ns: 20000000 ns
sched_min_granularity_ns: 4000000 ns
```

Figure 4. Durian analyzer configuration output

IV. Experimental Evaluation

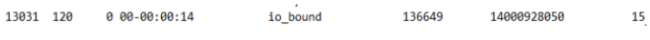
Our evaluation of Durian focuses on three aspects: (a) The correctness of the time statistics measured by Durian, (b) the impact of the execution of eBPF programs on the system performance, and (c) the impact of the resources used by Durian user space components to the system.

A. Correctness of time statistics measurements

We did manual end-to-end testing by analyzing the statistics produced by Durian to two programs that will behave as a fully CPU-bound process and an extremely I/O-bound process. The CPU-bound process will only do a busy infinite loop while the I/O-bound process will only invoke the sleep() system call infinitely. Both processes will be run for 15 seconds and stopped manually with a keyboard interrupt. There is no specific reason why 15 seconds is chosen for the experiments, it is just an arbitrary number of seconds where we have decided that it is long enough to have representative statistics but not too long because the statistics tend to converge at one point in time assuming that the system load stays relatively constant. This testing method is undesirable because manual interrupt timing is non-deterministic and it is almost impossible to guarantee that the manual interrupt happens exactly 15 seconds after the start of the process. However, the usage of a more deterministic approach such as with the help of timeout command line utility will affect the fair share that the task receives, our testing showed that timeout reduces the fair share of a fully CPU-bounded process by on average 60%.


```
21984  120     0 00-00:00:15          cpu_bound        14591059712                 0          19
```

Figure 5. CPU bound process measurement



```
13031  120     0 00-00:00:14          io_bound           136649        14000928050          15
```

Figure 6. I/O bound process measurement

Figure 5 and Figure 6 shows both of the measurement for each type of process is as expected. This experiment should only be considered as a sanity check rather than exact data. The reason for that is because the experiment involves human intervention which might contribute to the uncertainty of the measurement to a certain degree.

B. User space components benchmark

Durian user space components consist of Durian daemon, Redis, and the data analyzer. Benchmark will only be done for Durian daemon and Redis. Durian is used to benchmark both components as both the daemon and Redis is registered as normal task so it should be visible by Durian. The experiment is done on a machine with normal load so it is relatively idle.


```
 4544  120     0 00-00:00:14        redis-server       9946580542        4652814610         167
```

Figure 7. Redis scheduling statistics

```
13050  120     0 00-00:00:14             duriand       4937640856        9601443877       17048
```

Figure 8. Durian daemon scheduling statistics

As shown in both Figure 7 and Figure 8, both of the user space components use a relatively high CPU proportion. Durian daemon uses a high CPU proportion since it does a continuous poll of the ring buffer so it makes sense if it uses a high amount of CPU due to the system not having any other tasks to process. The Redis server however has no reason to use that much CPU outside of the fact that it has to process a significant amount of transactions by the request of the Durian daemon. This is undesirable because we want Durian to have as little footprint on the system resource.

C. Kernel space component benchmark

Similar to how the other two experiments are conducted the kernel space component will also be benchmarked for 15 seconds. Durian kernel space component only consists of the eBPF programs. Bpftool will be used to help benchmark the eBPF programs.

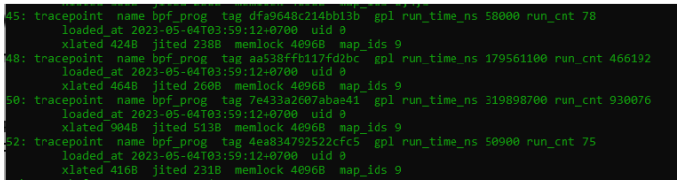


Figure 9. Durian probes statistics

Fortunately the benchmark shows a good result. Every entry in Figure 9 for id 45, 48, 50, and 52 represents a single eBPF program that is attached to a single tracepoint which is sched_wakeup_new, sched_wakeup, sched_switch, and sched_process_exit respectively. Programs that are attached to sched_wakeup and sched_switch have the highest number of invocations (run_cnt) hence they also use a higher proportion of the CPU (99.978%). However, it still uses a relatively small proportion of the CPU. Throughout the 15-second period of the benchmark, it only uses 0.3 seconds of the CPU time which is very small, especially on a multi-core system.

V. Conclusion

We presented Durian, an on-demand eBPF-based solution to monitor the Completely Fair Scheduler in order to determine the scheduling policy fairness of fair share allocation. Our implementation showed that tracepoints alone are enough to monitor the necessary task state transitions without the need for kprobe which is desirable as kprobe should be avoided due to it not having a stable ABI and its synchronous nature of kprobe invocation which will incur a direct overhead on the scheduling algorithm. We have also shown that the eBPF component of Durian incurs little to no effect on the system performance as it uses a very small proportion of the CPU but the user-space component still uses an undesirable amount of CPU and will need improvement or replacement, especially on the use of Redis server for Durian external store. Finally, the fair share fairness analysis has shown that processes that are I/O bounded tend to lose their fair share while more CPU-bounded processes will monopolize the CPU hence receiving even more fair share compared to their actual priorities.

References

[1] C. W. Wong, I. K. T. Tan, R. D. Kumari, and F. Wey, “Towards achieving fairness in the Linux scheduler,” Operating Systems Review, vol. 42, no. 5, pp. 34–43, Jul. 2008

[2] Y. Zhou, Z. Wang, S. Dharanipragada, and M. Yu, "Electrode: Accelerating Distributed Protocols with eBPF," in 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), pp. 1391-1407, 2023.

[3] Stevens, W. R. (2013). Advanced programming in the UNIX environment (3rd ed.). Upper Saddle River, NJ: Addison-Wesley.

[4] Love, R. (2010). Linux Kernel Development (3rd ed.). Indianapolis, IN: Sams.

[5] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux Scheduler: A Decade of Wasted Cores," in Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16), New York, NY, USA, 2016, pp. 1-16, doi: 10.1145/2901318.2901326.

[6] McCanne, S., & Jacobson, V. (1993). The BSD Packet Filter: A New Architecture for User-level Packet Capture. In Proceedings of the Winter 1993 USENIX Conference (pp. 259-268).

[7] S. Qi, L. Monis, Z. Zeng, I. C. Wang, and K. K. Ramakrishnan, "SPRIGHT: Extracting the Server from Serverless Computing! High-Performance eBPF-Based Event-Driven, Shared-Memory Processing," in Proceedings of the ACM SIGCOMM 2022 Conference, pp. 780-794, August 2022.

[8] eBPF Verifier — The Linux Kernel Documentation. https://docs.kernel.org/bpf/verifier.html. Last accessed: 2023-05-31.

[9] Linux Programmer’s Manual. bpf-helpers(7). https://man7.org/linux/man-pages/man7/bpf-helpers.7.html.. Last accessed: 2023-05-31.

[10] Gregg, B. (2019). BPF Performance Tools: Linux System and Application Observability. Beijing: O'Reilly Media, Inc.

[11] The Linux kernel development community. BPF Ring Buffer. https://www.kernel.org/doc/html/latest/bpf/ringbuf.html. Last accessed: 2023-05-31.

[12] Bpf ring buffer. https://nakryiko.com/posts/bpf-ringbuf. Last accessed: 2023-05-31.

[13] Humphries, et al. (2021). ghOst: Fast & Flexible User-Space Delegation of Linux Scheduling. In Proceedings of the 2021 ACM SIGOPS Operating Systems Review (pp. 588-694).

[14] Calavera, D., & Fontana, L. (2020). Linux Observability with BPF. O'Reilly Media, Inc.

[15] H. Marouani and M. R. Dagenais, “Internal Clock Drift Estimation in Computer Clusters,” Journal of Computer Systems, Networks, and Communications, vol. 2008, pp. 1–7, Jan. 2008, doi: 10.1155/2008/583162.